

OPTIONAL TASK

Name of Student : Anuja Vilas Wankhade

Domain: Artificial intelligence and machine Learning (AIML)

College Name: Manav School of Polytechnic AKOLA

Branch : Computer Engineering

GitHub Repositories Link: <https://github.com/anujaw193-star/python-project-collection>

Kaggle Dataset Link:

<https://www.kaggle.com/datasets/abbas829/ecommerce-sales-dataset>

1. Load the dataset.

Code:

```
[1] import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

[2] df = pd.read_csv("ecommerce_sales_analytics_5000.csv")
print(df.head(10))
print(df.tail())
print(df.shape)
print(df.info())
print(df.dtypes)
```

Output:

	order_id	order_date	customer_id	product_category	region	quantity	
0	10001	1/1/2022	1102	Beauty	South	7	
1	10002	1/2/2022	1435	Clothing	South	7	
2	10003	1/3/2022	1860	Beauty	East	3	
3	10004	1/4/2022	1270	Electronics	West	5	
4	10005	1/5/2022	1106	Clothing	West	5	
5	10006	1/6/2022	1071	Electronics	West	2	
6	10007	1/7/2022	1700	Clothing	South	6	
7	10008	1/8/2022	1020	Beauty	North	6	
8	10009	1/9/2022	1614	Clothing	South	6	
9	10010	1/10/2022	1121	Clothing	West	4	
	unit_price	discount	payment_method	delivery_days	customer_rating		
0	373.65	0.28	Wallet	10	4.7		
1	47.74	0.09	Card	6	3.9		
2	311.28	0.31	COD	6	2.5		
3	524.47	0.02	Wallet	6	1.6		
4	139.87	0.33	Wallet	4	4.9		
5	264.44	0.20	Card	3	3.3		
6	564.60	0.23	Card	1	1.8		
7	265.71	0.13	Card	2	3.7		
8	224.65	0.27	Card	2	1.4		
9	272.71	0.06	Card	10	4.5		

```

revenue
0 1883.20
1 304.10
2 644.35
3 2569.90
4 468.56
5 423.10
6 2608.45
7 1387.01
8 983.97
9 1025.39

order_id order_date customer_id product_category region quantity
4995 14996 9/5/2035 1289 Clothing West 2
4996 14997 9/6/2035 1294 Beauty North 3
4997 14998 9/7/2035 1450 Beauty West 3
4998 14999 9/8/2035 1903 Electronics South 4
4999 15000 9/9/2035 1214 Home East 7

unit_price discount payment_method delivery_days customer_rating
4995 371.74 0.27 COD 3 2.3
4996 553.67 0.14 COD 11 2.2
4997 77.22 0.24 Wallet 6 3.4
4998 138.41 0.21 COD 4 2.8
4999 231.58 0.34 COD 5 3.9

```

```

*** revenue
4995 542.74
4996 1428.47
4997 176.06
4998 437.38
4999 1069.90
(5000, 12)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 12 columns):
#   Column              Non-Null Count  Dtype
---  -
0   order_id             5000 non-null   int64
1   order_date           5000 non-null   object
2   customer_id          5000 non-null   int64
3   product_category     5000 non-null   object
4   region               5000 non-null   object
5   quantity             5000 non-null   int64
6   unit_price           5000 non-null   float64
7   discount             5000 non-null   float64
8   payment_method       5000 non-null   object
9   delivery_days        5000 non-null   int64
10  customer_rating      5000 non-null   float64
11  revenue              5000 non-null   float64
dtypes: float64(4), int64(4), object(4)

```

```

dtypes: float64(4), int64(4), object(4)
memory usage: 468.9+ KB
None
order_id             int64
order_date           object
customer_id          int64
product_category     object
region              object
quantity             int64
unit_price           float64
discount             float64
payment_method       object
delivery_days        int64
customer_rating      float64
revenue              float64
dtype: object

```

2. Missing & Duplicate Values

Code & Output:

```
[5]
# Missing Values
print(df.isnull().sum())

# Total Missing Values
print(df.isnull().sum().sum())

# Duplicate Values
print(df.duplicated().sum())

# Remove Duplicates
df = df.drop_duplicates()
print(df.shape)
```

```
... order_id      0
order_date      0
customer_id     0
product_category 0
region          0
quantity        0
unit_price      0
discount        0
payment_method  0
delivery_days   0
customer_rating 0
revenue         0
dtype: int64
0
0
(5000, 12)
```

3. Statistical Summary

Code:

```
[6]
print(df.describe())
numeric_columns = ['quantity', 'unit_price', 'delivery_days', 'customer_rating', 'revenue']
for col in numeric_columns:
    print("\n", col)
    print("Minimum :", df[col].min())
    print("Maximum :", df[col].max())
    print("Mean :", df[col].mean())
    print("Median :", df[col].median())
```

Output:

```
... count    order_id  customer_id  quantity  unit_price  discount \
mean    12500.500000  1505.701200    4.044800   308.418774    0.179984
std      1443.520003    290.836902    2.020398   169.259369    0.101404
min     10001.000000   1000.000000    1.000000    15.150000    0.000000
25%     11250.750000   1253.000000    2.000000   161.895000    0.090000
50%     12500.500000   1510.000000    4.000000   309.890000    0.180000
75%     13750.250000   1761.000000    6.000000   455.557500    0.270000
max     15000.000000   1999.000000    7.000000   599.960000    0.350000

      delivery_days  customer_rating  revenue
count    5000.000000    5000.000000  5000.000000
mean         6.118800         2.973980  1021.955148
std         3.153264         1.157722   825.584219
min         1.000000         1.000000   11.210000
25%         3.000000         2.000000   354.527500
50%         6.000000         3.000000   796.650000
75%         9.000000         4.000000  1515.690000
max        11.000000         5.000000  4119.330000

quantity
Minimum : 1
Maximum : 7
Mean : 4.0448
Median : 4.0
```

```

quantity
Minimum : 1
Maximum : 7
Mean : 4.0448
Median : 4.0

unit_price
Minimum : 15.15
Maximum : 599.96
Mean : 308.41877400000004
Median : 309.89

delivery_days
Minimum : 1
Maximum : 11
Mean : 6.1188
Median : 6.0

customer_rating
Minimum : 1.0
Maximum : 5.0
Mean : 2.9739800000000005
Median : 3.0

revenue
Minimum : 11.21
Maximum : 4119.33
Mean : 1021.955148
Median : 706.65

```

4. Histogram

Code:

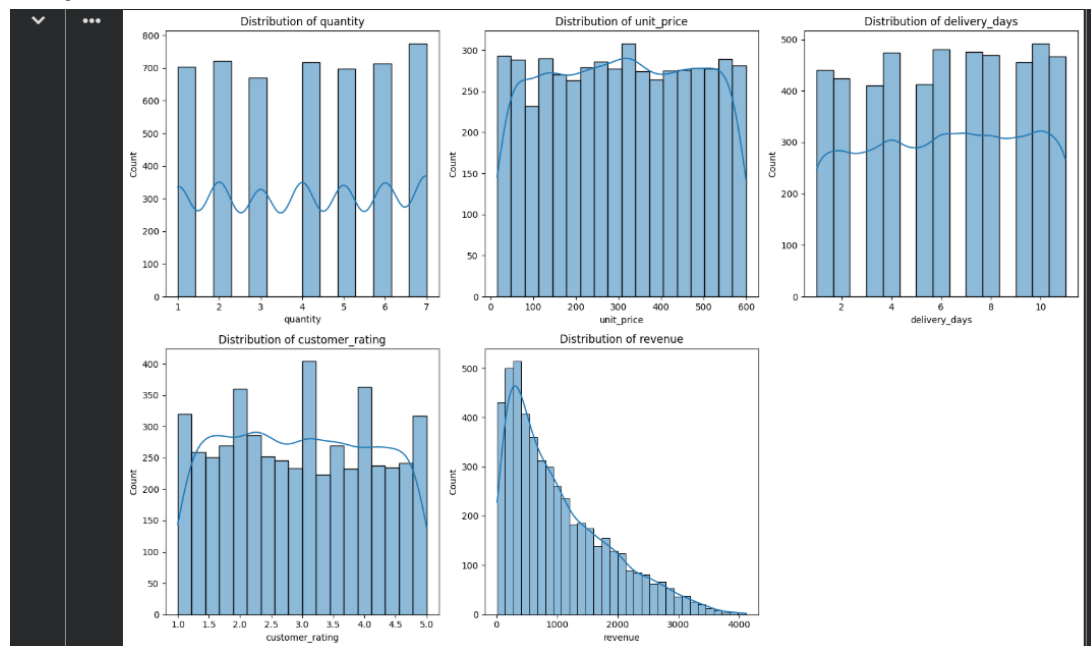
```

[7] ✓ 1s ▶ numeric_columns = ['quantity', 'unit_price', 'delivery_days', 'customer_rating', 'revenue']
plt.figure(figsize=(15, 10))
for i, col in enumerate(numeric_columns, 1):
    plt.subplot(2, 3, i)
    sns.histplot(df[col], kde=True)
    plt.title(f'Distribution of {col}')
    plt.xlabel(col)

plt.tight_layout()
plt.show()

```

Output:

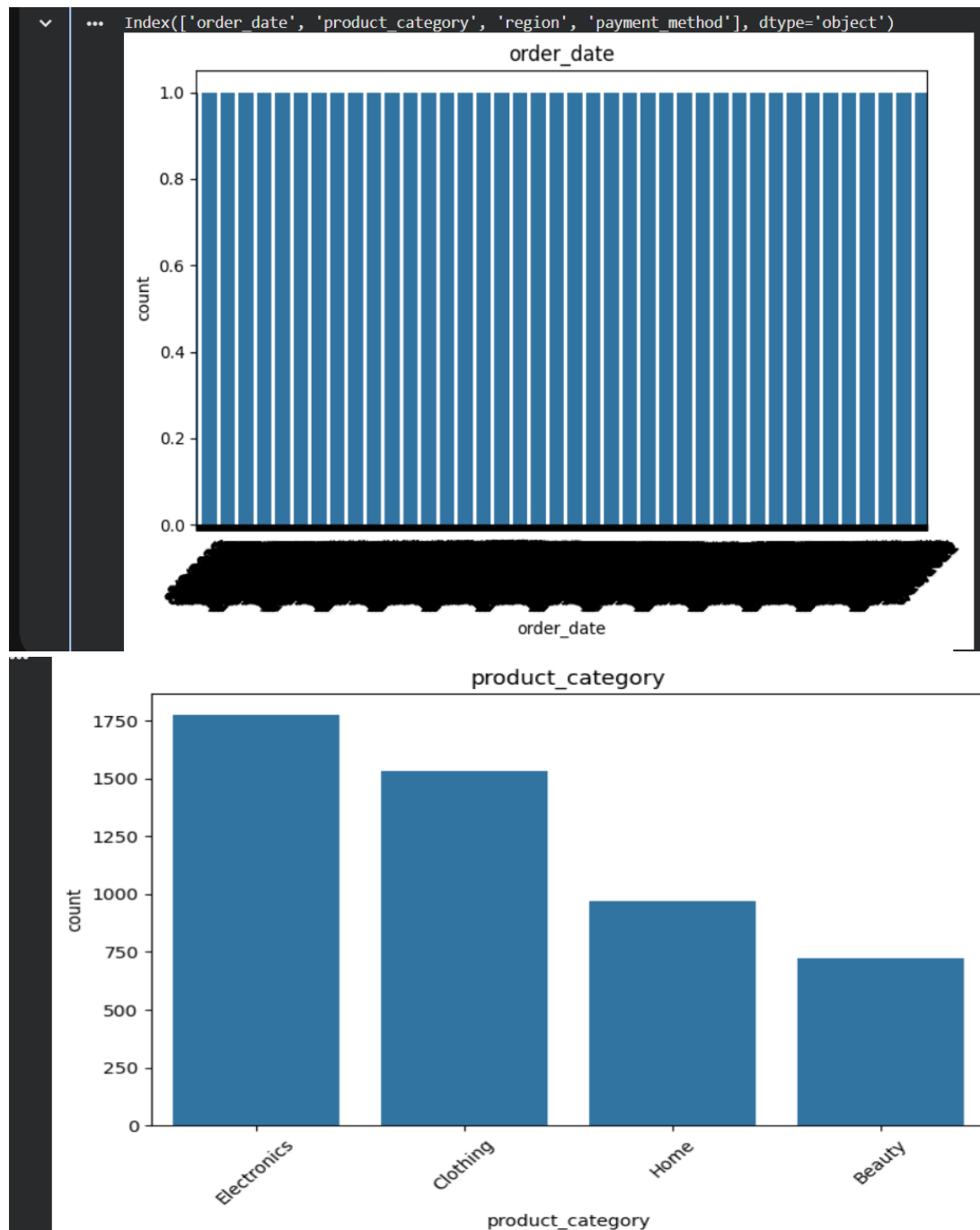


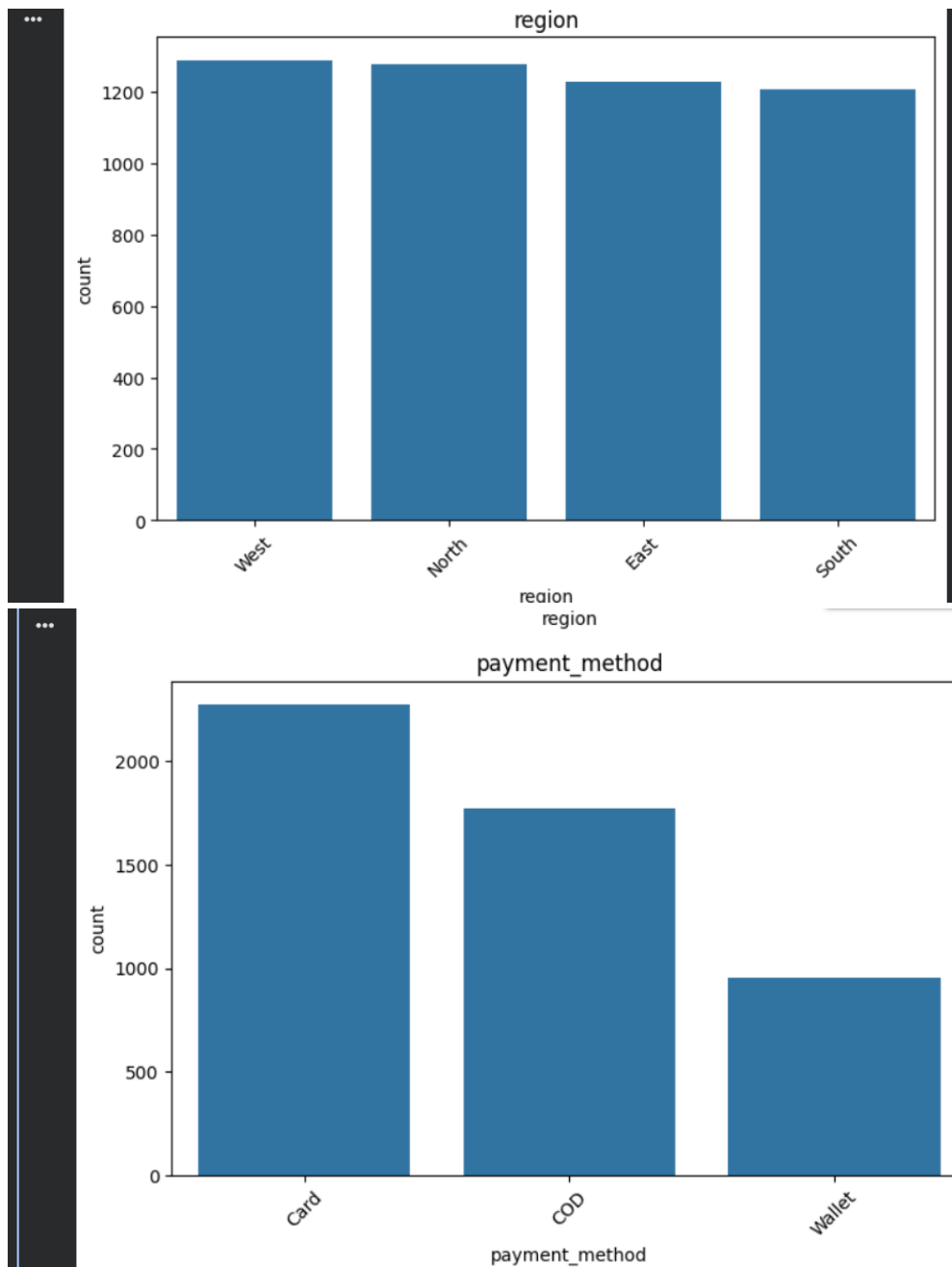
5. Count Plot

Code:

```
[8] ✓ 40s from pandas.core.arrays import categorical
categorical_columns=df.select_dtypes(include=['object']).columns
print(categorical_columns)
for col in categorical_columns:
    plt.figure(figsize=(8,5))
    sns.countplot(data=df,x=col,order=df[col].value_counts().index)
    plt.xticks(rotation=45)
    plt.title(col)
    plt.show()
```

Output:



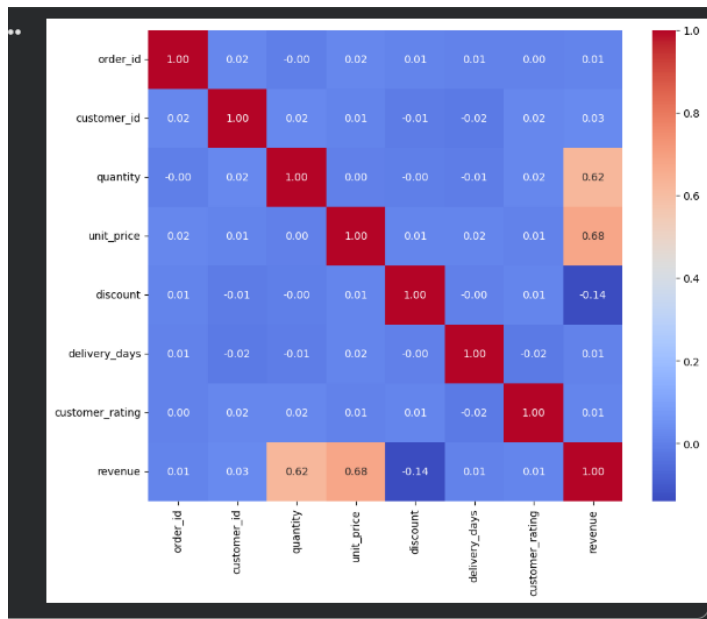


6. Heatmap

Code:

```
[10]
✓ 0s plt.figure(figsize=(10,8))
corr=df.select_dtypes(include=['int64', 'float64']).corr()
sns.heatmap(corr,annot=True,cmap='coolwarm', fmt='.2f')
plt.show()
```

Output:



7. Feature Identification

Code & Output:

```
[11]
✓ 0s
x=df.drop('revenue', axis=1)
y=df['revenue']
print(x.columns)
print(y.name)

... Index(['order_id', 'order_date', 'customer_id', 'product_category', 'region',
          'quantity', 'unit_price', 'discount', 'payment_method', 'delivery_days',
          'customer_rating'],
          dtype='object')
revenue
```

8. Standard Scaling

Code:

```
[9] ✓ 2s
from sklearn.preprocessing import StandardScaler
X=df_encoded.drop('revenue', axis=1)
y=df_encoded['revenue']
scaler=StandardScaler()
X_scaled=scaler.fit_transform(X)
X_scaled=pd.DataFrame(X_scaled, columns=X.columns)
print(X_scaled)
```

Output:

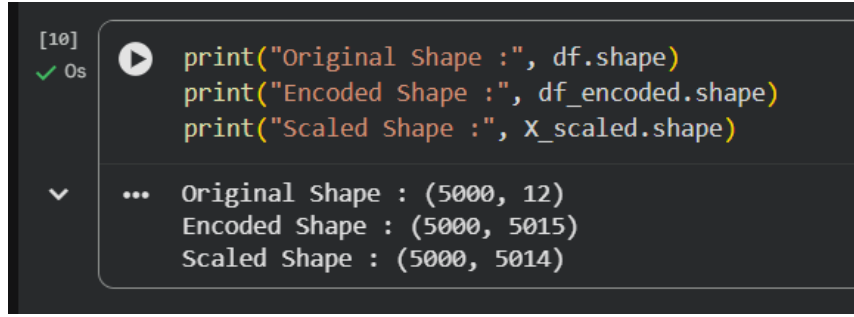
...	order_id	customer_id	quantity	unit_price	discount	delivery_da
0	-1.731704	-1.388206	1.462828	0.385431	0.986412	1.2309
1	-1.731012	-0.243120	1.462828	-1.540268	-0.887471	-0.0376
2	-1.730319	1.218326	-0.517177	0.016906	1.282289	-0.0376
3	-1.729626	-0.810505	0.472825	1.276578	-1.577850	-0.0376
4	-1.728933	-1.374451	0.472825	-0.995901	1.479540	-0.6720
...	...	...	...	...	...	.
4995	1.728933	-0.745170	-1.012179	0.374145	0.887787	-0.9891
4996	1.729626	-0.727976	-0.517177	1.449112	-0.394344	1.5481
4997	1.730319	-0.191540	-0.517177	-1.366080	0.591911	-0.0376
4998	1.731012	1.366190	-0.022176	-1.004528	0.296034	-0.6720
4999	1.731704	-1.003072	1.462828	-0.454016	1.578165	-0.3548
	customer_rating	order_date_1/1/2023	order_date_1/1/2024	\		
0	1.491025	-0.014144	-0.014144			
1	0.799944	-0.014144	-0.014144			
2	-0.409448	-0.014144	-0.014144			
3	-1.186914	-0.014144	-0.014144			
4	1.663795	-0.014144	-0.014144			
...	...	...	...			
4995	-0.582219	-0.014144	-0.014144			
4996	-0.668604	-0.014144	-0.014144			
4997	0.368018	-0.014144	-0.014144			
4998	-0.150293	-0.014144	-0.014144			
4999	0.799944	-0.014144	-0.014144			

	order_date_1/1/2025	...	order_date_9/9/2034	order_date_9/9/2035
0	-0.014144	...	-0.014144	-0.014144
1	-0.014144	...	-0.014144	-0.014144
2	-0.014144	...	-0.014144	-0.014144
3	-0.014144	...	-0.014144	-0.014144
4	-0.014144	...	-0.014144	-0.014144
...	...	...	...	...
4995	-0.014144	...	-0.014144	-0.014144
4996	-0.014144	...	-0.014144	-0.014144
4997	-0.014144	...	-0.014144	-0.014144
4998	-0.014144	...	-0.014144	-0.014144
4999	-0.014144	...	-0.014144	70.703607
	product_category_Clothing	product_category_Electronics	\	
0	-0.664332	-0.742529		
1	1.505271	-0.742529		
2	-0.664332	-0.742529		
3	-0.664332	1.346748		
4	1.505271	-0.742529		
...	...	...		
4995	1.505271	-0.742529		
4996	-0.664332	-0.742529		
4997	-0.664332	-0.742529		
4998	-0.664332	1.346748		
4999	-0.664332	-0.742529		

	product_category_Home	region_North	region_South	region_West
0	-0.490293	-0.585357	1.771743	-0.589360
1	-0.490293	-0.585357	1.771743	-0.589360
2	-0.490293	-0.585357	-0.564416	-0.589360
3	-0.490293	-0.585357	-0.564416	1.696755
4	-0.490293	-0.585357	-0.564416	1.696755
...	...	...	...	...
4995	-0.490293	-0.585357	-0.564416	1.696755
4996	-0.490293	1.708360	-0.564416	-0.589360
4997	-0.490293	-0.585357	-0.564416	1.696755
4998	-0.490293	-0.585357	1.771743	-0.589360
4999	2.039598	-0.585357	-0.564416	-0.589360
	payment_method_Card	payment_method_Wallet		
0	-0.911867	2.056727		
1	1.096651	-0.486209		
2	-0.911867	-0.486209		
3	-0.911867	2.056727		
4	-0.911867	2.056727		
...	...	...		
4995	-0.911867	-0.486209		
4996	-0.911867	-0.486209		
4997	-0.911867	2.056727		
4998	-0.911867	-0.486209		
4999	-0.911867	-0.486209		
[5000 rows x 5014 columns]				

9. Final Shape

Code & Output:



```
[10] ✓ Os  print("Original Shape :", df.shape)
print("Encoded Shape :", df_encoded.shape)
print("Scaled Shape :", X_scaled.shape)

⌵ ... Original Shape : (5000, 12)
Encoded Shape : (5000, 5015)
Scaled Shape : (5000, 5014)
```

Summary:

The E-commerce Sales Analytics dataset was analyzed using Python libraries such as Pandas, Matplotlib, Seaborn, and Scikit-learn. The dataset was successfully loaded and explored to understand its structure. Missing values and duplicate records were checked, and the dataset was found to be clean for analysis. Statistical analysis helped understand the distribution of numerical features.

Various visualizations, including histograms, count plots, and a heatmap, were used to identify data distribution and relationships between features. Categorical and numerical features were identified separately. Categorical variables were converted into numerical form using One-Hot Encoding, and numerical features were standardized using StandardScaler to prepare the data for machine learning.

After preprocessing, the original dataset contained 5,000 rows and 12 columns, while the encoded dataset expanded to 5,015 columns due to One-Hot Encoding. The final scaled dataset contained 5,000 rows and 5,014 features, making it suitable for machine learning algorithms.